Introduction au package samfr

Yves Croissant

C E M O I
Université de La Réunion

9 novembre 2026

Table des matières

1 Les tableaux de synthèse **2**

2 Extraction des tableaux de synthèse avec `stbl` **4**

3 Normalisation et transformation des séries **7**

4 Format des tableaux **10**

5 Construction d'une MCS avec la fonction sam **14**

6 Figures **20**

6.1 Structure des prélèvements obligatoires 20

6.2 Parts des prélèvements obligatoire dans le PIB 21

6.3 Structure productive 23

6.4 Revenus des facteurs par habitant en euros constants 24

samfr (pour Social Account Matrix for France) est un package permettant de construire facilement et de manière personnalisée une MCS (matrice de comptabilité sociale) pour la France. Le but de la MCS est de résumer, sous la forme d'un tableau à double entrée, l'activité économique d'un territoire. Elle est construite à partir des données des deux tableaux de synthèse :

— le TES (Tableau Entrées-Sorties),
— le TEE (Tableau Economique d'ensemble).

Le premier présente les emplois et les recettes en produits, la matrice des consommations intermédiaires, ainsi que les comptes de production et d'exploitation des branches productives. Le second synthétise l'activité des agents institutionnels (ménage, entreprise, Etat et reste du monde en particulier).

Le package **sam** propose deux fonctions principales :

— `stbl` permet d'extraire et de mettre en forme les éléments des tableaux de synthèse,
— `sam` met en forme les éléments des tableaux de synthèse afin de construire une matrice de comptabilité sociale.

1 Les tableaux de synthèse

Le TES se présente comme un tableau "en T", constitué de 4 éléments :

— à gauche les ressources en produits,
— à droite les emplois en produits,
— au milieu le tableau des entrées intermédiaires (le TEI),
— en bas les comptes de production et d'exploitation des branches.

Le TEI est un tableau carré indiquant les consommations intermédiaires utilisées par chaque branche (en colonne) pour chaque produit (en ligne).

Un quatrième tableau complète le TES, il est disponible dans le TES symétrique ; il s'agit du tableau de production, qui indique la production de chaque produit (en colonne) par chaque branche (en ligne).

sam contient l'ensemble des TES disponibles sur le site de l'INSEE, dans deux nomenclatures :

— la nomenclature **A17** en 17 branches depuis 1949,
— la nomenclature **A38** en 38 branches depuis 1978.

Le TEE présente les emplois (à gauche) et les ressources (à droite) pour les secteurs institutionnels (sociétés financières et non-financières, ménages, ISBL, Etat et reste du monde), en décrivant les opérations comme l'enchaînement d'un ensemble de comptes, le solde d'un compte (en emploi) devenant la ressource du compte suivant :

- le **compte de production** soustrait de la production les consommations intermédiaires et est soldé par la *valeur ajoutée brute*,
- le **compte d'exploitation** soustrait de la valeur ajoutée brute les salaires (incluant les cotisations salariales et patronales) et les impôts à la production et est soldé par l'*excédent brut d'exploitation* (les ressources en salaires et en impôts à la production ne sont pas intégrés). L'EBE des ménages est la somme du revenu mixte brut et de l'EBE "pur", constitué des loyers fictifs.
- le **compte d'affectation des revenus primaires** indique les transferts de revenus de la propriété entre agents (un jeu à somme nulle), les salaires (incluant les cotisations patronales et salariales) et les impôts à la production reçus. Le solde de ce compte est le *revenu primaire*.
- le **compte de distribution secondaire des revenus** intègre l'impôt sur le revenu, les cotisations et prestations sociales, ainsi que des transferts divers (en particulier les primes et indemnités d'assurance-dommage), son solde est le *revenu disponible*,
- le **compte d'utilisation du revenu** intègre la consommation finale et son solde est l'*épargne brute*.

Le TEE n'est utilisé qu'à partir du compte d'affectation des revenus primaires. Notons que :

- La consommation finale des ménages dans le TEE est nette de la correction territoriale (non sectorisée),
- l'épargne brute des ménages est augmentée de D8 (ajustement pour variation des droits à pension), au détriment de celle des sociétés non financières.

Les tableaux de synthèse sont stockés dans un tableau interne de **sam** appelé `TBL` :

```
samfr:::TBL |> head()
```

```
    an  tableau compte compte2 variable valeur sgn
1 1949 produits     AZ    none       cf  0.609 neg
2 1949 produits     AZ    none     fbcf  0.041 neg
3 1949 produits     AZ    none       ds  0.001 neg
4 1949 produits     AZ    none        x  0.080 neg
5 1949 produits     AZ    none    ci.pr  3.612 neg
6 1949 produits     AZ    none     y.pr  3.456 pos
```

Les variables de ce tableau sont :

- an : l'année,
- `tableau` : `"produits"` pour les emplois et les ressources en produits, `"branches"` pour les comptes de production et d'exploitation des branches, `"tei"` pour la table des consommations intermédiaires et `"matprod"` pour la table de production,

- `compte` et `compte2` indiquent le(s) compte(s) concernés; pour `tee`, il s'agit des agents, pour `branches` des branches et pour `produits` des produits. `compte2` n'est pertinent que pour les deux tableaux matriciels des emplois intermédiaires et de la production. Dans ce cas, `compte` indique le produit et `compte2` la branche,
- `variable` est l'acronyme de la variable, par exemple `"cf"` pour la consommation finale des ménages,
- `valeur` est la valeur de la variable en milliards d'euros courants,
- `sgn` est le "signe" de la variable : `"pos"` s'il s'agit d'une ressource, `"neg"` s'il s'agit d'un emploi et `"both"` pour les deux tableaux matriciels.

Toutes les lignes de `sam:::TBL` sont exactement des valeurs de variables disponibles sur le site de l'INSEE, à une exception près. La matrice de production n'est disponible que pour 2023, pour les autres années, nous ne connaissons que les marges de cette matrice, c'est-à-dire la production des branches et la production en produit. La matrice a été estimée pour chaque année par la méthode des facteurs de croissance, la matrice initiale pour l'année t étant celle obtenue en $t + 1$.

Toutes les valeurs sont exprimées en milliards d'euros courant.

Les tableaux renvoyés par `sam` et par `stbl` sont de classe `"tbl"`, `"tbl_df"` et `"data.frame"`, ce sont autrement dit des tibbles et pas des data frame ordinaires. L'utilisation du package **tibble** est conseillé car cela permet une impression beaucoup plus agréables des tableaux, mais elle est optionnelle. Nous l'utiliserons dans ce document, par conséquent, nous chargerons, en plus du package **sam**, le package **tibble**, et également le package **tinytable** qui permet la mise en forme des tableaux.

```
library(samfr)
library(tinytable)
library(tibble)
```

2 Extraction des tableaux de synthèse avec `stbl`

`sam:::TBL` n'a pas vocation a être manipulé directement par l'utilisateur. La fonction `sam::stbl` permet d'extraire et de mettre en forme un sous-ensemble de lignes de ce tableau. Ces deux arguments principaux sont l'année (`an`) et le tableau (`tableau`). Par exemple, pour extraire les comptes des branches en 2013, on utilisera :

```
stbl(an = 2013, tableau = "branches")
```

```
# A tibble: 183 x 4
```

```
  naf   variable sgn   valeur
* <chr> <chr>    <chr>  <dbl>
1 AZ    ci.br    neg    55.3
2 BZ    ci.br    neg     4.01
3 CA    ci.br    neg   119.
# i 180 more rows
```

Les arguments supplémentaires sont :

- `unit`, l'unité de mesure, soit `"G"` pour des milliards d'euros, `"M"` pour des millions d'euros, `"K"` pour des milliers d'euros et `"E"` pour des euros,
- `sgn` permet de spécifier si on souhaite renvoyer les emplois (`sig = "neg"`) ou les ressources (`sig = "pos"`), les deux étant renvoyés par défaut,
- `variables` est un vecteur de caractères permettant de renvoyer un sous-ensemble des variables du tableau sélectionné,
- `net_tax` est une valeur logique, si elle est vraie, les taxes sur les produits (`t.pr`) dans les ressources en produits et les taxes sur la production (`t.br`) dans le compte d'exploitation des branches sont renvoyées nettes des subventions (`s.pr` et `s.br`). Dans le cas inverse, le tableau renvoie séparément les taxes et les subventions,
- `isbl` est une valeur logique, si elle est vraie, la consommation finale des ISBL (institutions sans but lucratif) est indiquée sous le nom de `"isbl"` dans la table des produits et les ISBL constituent un agent du TEE ; autrement les ISBL sont fusionnés avec les ménages,

Par exemple, pour renvoyer les ressources en produits en millions d'euros courants en 2013 et 2015, en distinguant impôts et subventions :

```
stbl(an = c(2013, 2015), tableau = "produits", sgn = "pos",
       net_tax = FALSE, unit = "M")
```

```
# A tibble: 347 x 4
  an    naf   variable valeur
* <chr> <chr> <chr>     <dbl>
1 2013  AZ    m         12270
2 2015  AZ    m         13217
3 2013  BZ    m         55907
# i 344 more rows
```

Certaines variables sont spécifique au tableau. Pour le TES,

- `net_acc` est une valeur logique, si elle est vraie, la formation brute de capitale fixe (`"fbcf"`) est renvoyée nette de la variation des stocks ; autrement, la table des produits renvoie séparément la FBCF et la variation des stocks (`"ds"`),

— `naf` indique la nomenclature d'activité; par défaut, c'est `"A17"` de 1949 à 1977 et `"A38"` à partir de 1978.

```
stbl(2023, tableau = "produits", net_acc = FALSE, naf = "A4")
```

```
# A tibble: 42 x 4
  naf   variable sgn   valeur
* <chr> <chr>    <chr>  <dbl>
1 A     cf       neg     36.3
2 I     cf       neg    643.
3 R     cf       neg    770.
# i 39 more rows
```

On a ici choisi une nomenclature en 4 positions et la variation de stocks (`"ds"`) est renvoyée distinctement de l'investissement (`"fbcf"`). Notez que les deux variables `an` et `tableau` ne sont pas spécifiées et prennent donc les valeurs par défauts, respectivement 2023 et `"produits"`.

Pour le TEE, les arguments spécifiques sont :

— `sep_ent` : un booléen, faux par défaut qui indique si on souhaite distinguer ou fusionner les sociétés financières et non financières ; dans le premier cas les deux agents sont `"snf"` et `"sf"`, dans le second cas, l'agent unique est `"ent"`,
— `agents` : un éventuel sous-ensemble d'agents,
— `comptes` : un sous ensemble de comptes parmi `"prod"` (compte de production), `"expl"` (compte d'exploitation), `"affect"` (compte d'affectation des revenus primaires), `"dist"` (compte de distribution des revenus secondaires) et `"util"` (compte d'utilisation du revenu),
— `net_cap` : un booléen (vrai par défaut), s'il est faux on distingue l'EBE, le revenu mixte brut et le solde pour chaque agent des revenus de la propriété, autrement un revenu du capital agrégé est renvoyé,
— `simpl_dist` : un booléen (faux par défaut), qui sera expliqué ci-dessous.

Par exemple, pour renvoyer les informations concernant les comptes de production et de distribution pour les sociétés financières et non-financières, on utilisera :

```
stbl(tableau = "tee", comptes= c("prod", "util"),
       agents = c("snf", "sf"), sep_ent = TRUE)
```

```
# A tibble: 10 x 5
  cpte  variable agent sgn   valeur
* <chr> <chr>    <chr> <chr>  <dbl>
```

```
1 prod  y.ag     sf    pos     288.
2 prod  y.ag     snf   pos    3810.
3 prod  ci.ag    sf    neg     198.
# i 7 more rows
```

3 Normalisation et transformation des séries

En particulier dans le cas où on s'intéresse à l'évolution chronologique d'une ou plusieurs variables, il peut être pertinent de mesurer cette variable par habitant et en euros constant. **sam** propose deux fonctions `pop` et `cpi` pour réaliser cette opération. Considérons par exemple les revenus du travail et du capital pour la période 2020-2021 :

```
rev_facteurs <- stbl(2020:2021, "branches",
                     variables = c("k", "l"), naf = "A1")
```

La fonction `pop` prend pour argument un tableau et renvoie soit le tableau augmenté de la colonne de population (en millions d'habitants) si `deflate = FALSE` ou la série divisée par la population si `deflate = TRUE` (par défaut) :

```
rev_facteurs |> pop(deflate = FALSE)
```

```
# A tibble: 4 x 5
  an    variable sgn   valeur   pop
  <chr> <chr>    <chr>  <dbl> <dbl>
1 2020  k        neg     806.  65.3
2 2021  k        neg     886.  65.5
3 2020  l        neg    1195.  65.3
# i 1 more row
```

```
rev_facteurs |> pop()
```

```
# A tibble: 4 x 4
  an    variable sgn   valeur
  <chr> <chr>    <chr>  <dbl>
1 2020  k        neg     12.4
2 2021  k        neg     13.5
3 2020  l        neg     18.3
# i 1 more row
```

La fonction `ipc` réalise la même opération avec l'indice des prix à la consommation. Elle a un argument supplémentaire `base` qui indique l'année de base souhaitée. Par exemple, pour obtenir les revenus des facteurs par habitant en euros de 1995, on utilisera :

```
rev_facteurs |> pop() |> ipc(base = 1995)
```

```
# A tibble: 4 x 4
  an    variable sgn   valeur
  <chr> <chr>    <chr>  <dbl>
1 2020  k        neg     13.9
2 2021  k        neg     15.0
3 2020  l        neg     20.6
# i 1 more row
```

Très souvent, il est utile d'agréger les valeurs par une fonction, selon un ou plusieurs caractères. Reprenons l'exemple du revenu des facteurs, cette fois-ci en nomenclature A4 :

```
rev_facteurs <- stbl(2020:2021, "branches",
                     variables = c("l", "k"),
                     naf = "A4")
rev_facteurs
```

```
# A tibble: 16 x 5
  an    naf   variable sgn   valeur
* <chr> <chr> <chr>    <chr>  <dbl>
1 2020  A     k        neg     31.5
2 2021  A     k        neg     35.4
3 2020  I     k        neg    146.
# i 13 more rows
```

Pour calculer par exemple le revenu total par secteur et par année, on doit sommer les valeurs en fonction des deux variables `an` et `naf`. Cette opération est assez malaisée à réaliser en base-R, mais évidente avec **dlyr**, puisqu'elle s'écrit :

```
summarise(rev_facteurs, valeur = sum(valeur), .by = c(an, naf))
```

sam propose la fonction `condsummary` qui prend pour argument le tableau, un vecteurs de caractères indiquant les variables de conditionnement et la fonction à utiliser, cette fonction étant appliquée à la colonne `"valeur"` :

```
condsummary(rev_facteurs, c("an", "naf"), sum)
```

```
# A tibble: 8 x 3
  an    naf   valeur
* <chr> <chr>  <dbl>
1 2020  A       40.9
2 2021  A       44.7
3 2020  I      370.
# i 5 more rows
```

Par défaut, la fonction renvoie un tableau contenant les variables de conditionnement et la série calculée. Si l'argument `dim` est fixé à `"table"`, `condsummary` renvoie la série calculée, avec un format compatible avec le tableau initial :

```
condsummary(rev_facteurs, c("an", "naf"), sum, dim = "table")
```

```
 [1]   40.9271   44.6706  370.2827  397.9315 1124.7987
 [6] 1236.5419  465.2047  492.5284   40.9271   44.6706
[11]  370.2827  397.9315 1124.7987 1236.5419  465.2047
[16]  492.5284
```

ce qui s'écrit avec **dplyr** :

```
mutate(rev_facteurs, valeur = sum(valeur), .by = c("an", "naf"))
```

Ce calcul est très utile pour calculer des parts, ici les parts des deux facteurs dans le revenu de chaque branche :

```
rev_facteurs$valeur /
    condsummary(rev_facteurs, c("an", "naf"), sum, dim = "table")
```

```
 [1] 0.7705628 0.7935555 0.3929868 0.3900139 0.4601286
 [6] 0.4634344 0.2398869 0.2483784 0.2294372 0.2064445
[11] 0.6070132 0.6099861 0.5398714 0.5365656 0.7601131
[16] 0.7516216
```

Ce qui s'écrit avec **dplyr** :

```
  mutate(rev_facteurs, valeur = valeur / sum(valeur),
         .by = c(an, naf)) |> pull(valeur)
```

Les modalités des différentes variables sont des noms courts (par exemple `"cf"` pour la consommation finale des ménages). Ces noms courts peuvent être remplacés par des noms long en utilisant la fonction `add_label` en renseignant en second argument les colonnes concernées :

```
  rev_facteurs |> add_label(c("variable", "sgn"))
```

```
# A tibble: 16 x 5
  an    naf   variable sgn     valeur
  <chr> <chr> <chr>    <chr>    <dbl>
1 2020  A     Capital  emplois   31.5
2 2021  A     Capital  emplois   35.4
3 2020  I     Capital  emplois  146.
# i 13 more rows
```

4 Format des tableaux

`stbl` renvoie un tableau au format “long”, c’est-à-dire que chaque ligne correspond à la valeur d’une variable, définie par un compte (agent ou code d’activité), un “signe” (emploi ou ressource), une année. Notez que lorsqu’une des colonnes est inutile (par exemple la colonne année si une seule année est sélectionnée), elle est supprimée. Ce format se prête bien aux différentes manipulations nécessaires à la construction d’une MCS.

En revanche, en termes d’apparence et d’analyse de résultats, un format “large” est préférable. Dans ce cas, la table initiale est pivotée en largeur, les modalités contenues dans une colonne du tableau au format “long” constituant autant de colonnes dans le tableau au format “large”. De plus des groupes de lignes et de colonnes peuvent être définis. Par exemple, pour un TEE, les groupes de lignes seront définies par les comptes (de production, d’exploitation, etc.) et les groupes de colonnes par les emplois / ressources. La fonction `xtbl` réalise cette opération. Ces deux arguments obligatoires sont un tableau et une formule indiquant la forme souhaitée pour le tableau formaté. Une formule du type `w + x ~ y + z` indique que les lignes correspondent aux modalités de `x`, les colonnes aux modalités de `z` et que les groupes de lignes et de colonnes sont définis respectivement par les modalités des variable `w` et `y`. Les groupes sont optionnels, par exemple des formules du type `w + x ~ z` ou `x ~ y + z` ou `x ~ z` sont tout à fait valables. Par exemple, pour présenter les comptes des branches, en se limitant aux emplois, on utilisera :

```
stbl(2023, naf = "A4", tableau = "branches", sgn = "neg") |>
    xtbl(naf ~ variable)
```

```
  naf     ci.br        k        l    t.br
1   A   68.9832  38.8742  10.3831 -7.2900
2   I 1154.4891 227.9250 269.0069 14.9826
3   R 1336.6982 613.0835 763.4103 67.7919
4   U  211.3586 122.6636 406.4558 11.1635
```

Pour le TEE, les colonnes sont les agents, en distinguant emplois et ressources, et les lignes sont les variables, regroupées par compte :

```
stbl(tableau = "tee", agents = c("men", "etat"),
     comptes = c("prod", "expl")) |>
    xtbl(cpte + variable ~ sgn + agent)
```

```
   cpte variable neg|men neg|etat  pos|men pos|etat
1  prod     y.ag      NA       NA  588.958  623.109
3  prod    ci.ag  86.554  157.696       NA       NA
5  prod      vab 502.404  465.413       NA       NA
7  expl      vab      NA       NA  502.404  465.413
9  expl     l.ag  76.296  346.293 1480.821       NA
12 expl  t.pr.ag      NA       NA       NA  318.565
13 expl  t.br.ag  28.890   13.006       NA  128.015
16 expl  s.pr.ag      NA       NA       NA  -31.819
17 expl  s.br.ag  -4.809   -0.972       NA  -38.133
20 expl      ebe 402.027  107.086       NA       NA
22 expl      rmb 135.433       NA  135.433       NA
```

Notez que les colonnes regroupées ont un nom constitué de la modalité de la variable de regroupement, le signe "|" et la modalité de la variable en colonne, par exemple "neg|men".

L'argument `shares` de `xtbl` permet de calculer des proportions en indiquant par rapport à quelles variables on souhaite calculer ces proportions. Par exemple, pour calculer la structure par branche des différents éléments des comptes de production et d'exploitation, on utilisera :

```
stbl(2022:2023, naf = "A4", tableau = "branches", sgn = "neg") |>
    xtbl(an + naf ~ variable, shares = c("an", "naf"))
```

```
    an naf     ci.br          k          l         t.br
1 2022   A 0.5916466 0.38582092 0.08943322 -0.066900719
3 2022   I 0.7417195 0.09136356 0.15743468  0.009482275
5 2022   R 0.4803208 0.22334013 0.27245502  0.023884057
7 2022   U 0.2736568 0.17094635 0.54081964  0.014577188
2 2023   A 0.6217475 0.35037427 0.09358317 -0.065704977
4 2023   I 0.6928028 0.13677659 0.16142962  0.008990979
6 2023   R 0.4806566 0.22045561 0.27451087  0.024376948
8 2023   U 0.2811960 0.16319429 0.54075753  0.014852160
```

Alors que pour calculer la part sectorielle de chaque composante des comptes de production et d'exploitation, on utilisera :

```
stbl(2022:2023, naf = "A4", tableau = "branches", sgn = "neg") |>
    xtbl(an + naf ~ variable, shares = c("an", "variable"))
```

```
    an naf      ci.br          k           l        t.br
1 2022   A 0.02411491 0.04748721 0.007253935 -0.09130016
3 2022   I 0.43817152 0.16298406 0.185078681  0.18755752
5 2022   R 0.46613671 0.65451008 0.526172640  0.77608183
7 2022   U 0.07157685 0.13501866 0.281494744  0.12766080
2 2023   A 0.02488994 0.03877547 0.007164434 -0.08413351
4 2023   I 0.41655312 0.22734611 0.185617228  0.17291340
6 2023   R 0.48229629 0.61152637 0.526760108  0.78238274
8 2023   U 0.07626065 0.12235205 0.280458230  0.12883736
```

La fonction `xtbl2tt` permet de formater un tableau issu de la fonction `xtbl` en **markdown**, **LaTeX** ou en **html**, en utilisant **tinytable**. La Table 1 présente les comptes de production et d'exploitation du TEE, avec des groupes de ligne correspondant aux comptes et des groupes de colonnes aux emplois et aux ressources. Les deux arguments optionnels de `xtbl2tt`, `height` pour la hauteur des lignes et un booléen `theme`, vrai par défaut. Si ce dernier argument est fixé à `FALSE`, un tableau brut est obtenu.

```
stbl(tableau = "tee", agents = c("men", "etat"),
     comptes = c("prod", "expl")) |>
    xtbl(cpte + variable ~ sgn + agent) |>
    xtbl2tt(height = 0.5, theme = FALSE)
```

En fixant `theme` à TRUE, le thème par défaut de **samfr** est appliqué. Dans le Table 2, on utilise également la fonction `label` pour remplacer les noms courts par des noms longs.

Tableau 1 – Un TEE formaté à l'aide de la fonction xtbl2tt sans thème

	neg		pos	
variable	men	etat	men	etat
prod				
y.ag			588.958	623.109
ci.ag	86.554	157.696		
vab	502.404	465.413		
expl				
vab			502.404	465.413
l.ag	76.296	346.293	1480.821	
t.pr.ag				318.565
t.br.ag	28.890	13.006		128.015
s.pr.ag				-31.819
s.br.ag	-4.809	-0.972		-38.133
ebe	402.027	107.086		
rmb	135.433		135.433	

```
stbl(tableau = "tee", agents = c("men", "etat")) |>
    xtbl(cpte + variable ~ sgn + agent,
         labels = c("cpte", "variable", "sgn", "agent")) |>
    xtbl2tt(height = 0.5)
```

Le thème appliqué correspond à la fonction `theme_sam`, qui peut être utilisée explicitement avec différents arguments, comme illustré par le Table 3 qui présente les comptes des branches pour trois année.

```
stbl(2021:2023, tableau = "branches", naf = "A4") |>
    xtbl(an + naf ~ sgn + variable, label = c("sgn", "naf")) |>
    xtbl2tt(height = 0.5, theme = FALSE) |>
    theme_sam(digits = 4, grid = FALSE,
              colors = list(groupes = "lightblue",
                            comptes = "lightyellow"))
```

`theme_sam` utilise en fait les deux fonctions de personnalisation de **tinytable** `format_tt` et `style_tt` qui peuvent être utilisées directement pour personnaliser avec plus de détails le tableau.

5 Construction d'une MCS avec la fonction sam

La fonction `sam` construit la MCS. Les différents éléments de la MCS sont obtenus en faisant appel à la fonction `stbl`, avec un format particulier appelé `"sam"`, obtenu en appliquant la fonction `stbl2sam` :

```
stbl(2023, tableau = "produits") |> stbl2sam() |> head()
```

```
  row_1 row_2 col_1 col_2   valeur
1    pr    AZ   agt   men  36.3330
2    pr    BZ   agt   men   0.0181
3    pr    CA   agt   men 208.8024
4    pr    CB   agt   men  51.5698
5    pr    CC   agt   men   9.2156
6    pr    CD   agt   men  59.4497
```

Chaque ligne contient toujours une valeur mais, le compte en ligne (ressource) et en colonne (emploi) est désormais identifié par deux variables :

Tableau 2 – Un TEE formaté à l'aide de la fonction xtbl2tt avec thème par défaut

	emplois		ressource	
variable	Ménages	Etat	Ménages	Etat
Compte de production				
Production			588.96	623
Consommation intermédiaire	86.6	157.7		
Valeur ajoutée brute	502.4	465.4		
Compte d'exploitation				
Valeur ajoutée brute			502.4	465
Rémunération des salariés	76.3	346.3	1480.82	
Impôts sur les produits				319
Autres impôts sur la production	28.9	13		128
Subventions sur les produits				-32
Autres subventions sur la production	-4.8	-1		-38
Excédent brut d'exploitation	402	107.1		
Revenu mixte brut	135.4		135.43	
Compte d'affectation des revenus primaires				
Excédent brut d'exploitation			402.03	107
Revenus de la propriété	63.6	55.2	200.43	22
Solde des revenus primaires bruts	2019.7	450.2		
Compte de distribution secondaire				
Solde des revenus primaires bruts			2019.72	450
Impôts courants sur le revenu et le patrimoine	273.3	0.2		357
Cotisations sociales nettes	542.9		0.01	462
Prestations sociales	0.6	529.7	591.75	
Autres transferts courants	96.7	579	150.63	505
Revenu disponible brut	1848.7	665.2		
Compte d'utilisation du revenu				
Revenu disponible brut			1848.66	665
Dépense de consommation finale	1544.8	682.1		
Epargne brute	310.1	-16.9		

TABLEAU 3 – Equilibre emplois-ressources formaté à l'aide de xtbl2tt

	emplois				ressource
naf	ci.br	k	l	t.br	y.br
	2021				
Agriculture, sylviculture et pêche	57.61	35.45	9.222	-7.968	94.31
Industrie	960.92	155.2	242.733	5.565	1364.41
Services marchands	1108.34	573.06	663.486	34.045	2378.93
Services non marchands	181.25	122.33	370.195	9.45	683.23
	2022				
Agriculture, sylviculture et pêche	66.15	43.13	9.998	-7.479	111.8
Industrie	1201.87	148.04	255.104	15.365	1620.38
Services marchands	1278.57	594.51	725.253	63.577	2661.92
Services non marchands	196.33	122.64	388	10.458	717.43
	2023				
Agriculture, sylviculture et pêche	68.98	38.87	10.383	-7.29	110.95
Industrie	1154.49	227.92	269.007	14.983	1666.4
Services marchands	1336.7	613.08	763.41	67.792	2780.98
Services non marchands	211.36	122.66	406.456	11.164	751.64

- `row_1` et `col_1` indiquent le groupe de comptes, soit `br` (branches), `pr` (produits), `fact` (facteurs), `mg` (marges), `tax` (taxes sur les branches et sur les produits), `dist` (distribution), `agt` (agent) et `acc` (accumulation),
- `row_2` et `col_2` détaille le compte, par exemple `men` pour les ménages, `t.br` pour les impôts sur la production.

L'argument `simpl_dist` de la fonction `stbl` n'a pas d'effet sur le tableau renvoyé par celle-ci, mais il est important si `stbl2sam` est appelé ensuite. Si cet argument est TRUE, le compte de distribution du TEE est simplifié :

- tous les impôts sont versés à l'Etat,
- toutes les cotisations sociales sont acquittées par les ménages et versées à l'Etat,
- toutes les prestations sociales sont versées aux ménages et acquittés par l'Etat,
- l'IR est acquitté par les ménages et par les entreprises.

L'argument principal de `sam` est l'année pour lequel on souhaite construire la MCS :

```
sam(2023)
```

```
# A tibble: 665 x 5
  row_1 row_2 col_1 col_2 valeur
  <chr> <chr> <chr> <chr>  <dbl>
1 pr    AZ    agt   men     36.3
2 pr    C1    agt   men    209.
3 pr    C2    agt   men     59.4
# i 662 more rows
```

Des arguments complémentaires sont passés à `stbl` :

- `naf` pour définir la nomenclature,
- `isbl` pour isoler les ISBL,
- `net_tax` pour agréger les taxes et les subventions sur la production et sur les produits,
- `net_acc` pour agréger la FBCF et la variation des stocks,
- `unit` pour définir l'unité,
- `sep_ent` pour séparer les sociétés financières et non-financières,
- `net_cap` pour détailler les revenus du capital.

D'autres sont spécifiques à `sam` :

- `rev_int` permet de calculer deux revenus intermédiaires des agents, le revenu primaire et le revenu disponible,
- `trace` permet d'imprimer des résultats intermédiaires,

— `tot_gen` permet d'ajouter un total pour chaque compte.

Une matrice de comptabilité sociale en 4 branches, avec mention des revenus intermédiaires, identification des ISBL et séparation de la variation des stocks et de la FBCF est obtenue en écrivant :

```
sam(2023, naf = "A4", rev_int = TRUE, isbl = TRUE, net_acc = FALSE)
```

```
# A tibble: 193 x 5
  row_1 row_2 col_1 col_2 valeur
  <chr> <chr> <chr> <chr>  <dbl>
1 pr    A     agt   men     36.3
2 pr    I     agt   men    643.
3 pr    R     agt   men    742.
# i 190 more rows
```

La fonction `sam2tt` permet de transformer le tableau contenant les différents éléments de la MCS dans un tableau au format "large", regroupant les lignes et les colonnes. Un exemple est donné dans le Table 4, pour lequel il n'y a pas de décomposition sectorielle et une simplification du TEE. Comme pour la fonction `xtbl2tt`, `sam2tt` a un argument `theme`, vrai par défaut qui applique la fonction `theme_sam` pour personnaliser le tableau :

```
sam(2023, naf = "A1", tot_gen = TRUE,
    rev_int = TRUE, simpl_dist = TRUE) |>
    sam2tt(height = 0.4, theme = FALSE) |>
    theme_sam(fontsize = 0.4)
```

TABLEAU 4 – Matrice de comptabilité sociale pour la France en 2023

	br	pr	fact		tax	RPB	dist				RDB	agt				acc		total
	A	A	l	k	tva	RPB	ir	cot	prest	atc	RDB	ent	etat	men	rdm	fbcf	acc	total
branches																		
A		5310																5310
produits																		
A	2772												682	1559	908	648	648	6569
facteurs																		
l	1449														34			1483
k	1003																	1003
marges																		
mc		0																0
mt		0																0
taxes																		
t.br	87																	87
t.pr		82																82
tva		206																206
transferts																		
ir												78		273				351
cot														543				543
prest													601					601
atc												168	579	97	37			880
agents																		
ent				422		422				143	319							565
etat				74	206	280	351	543		505	499							1848
men			1481	539		2020			601	151	1858							2772
rdm		971	2	-32		-30				81	15							1022
accumulation																		
acc												319	-14	300	44			648
total																		
total	5310	6569	1483	1003	206	2692	351	543	601	880	2692	565	1848	2772	1022	648	648	23971

6 Figures

Dans cette section, on donne quelques exemples de graphiques, réalisés avec le package **tinyplot**.

```
library(tinyplot)
```

6.1 Structure des prélèvements obligatoires

Les prélèvements obligatoires comprennent l'impôt sur le revenu et le patrimoine, les taxes sur les produits (en particulier TVA et droits de douane) et sur la production, ainsi que les cotisations sociales. On peut les extraire dans le TEE, en indiquant le nom des variables, qu'il s'agit d'une ressource et en limitant l'extraction aux opérations de l'Etat :

```
po <- stbl(1949:2023, "tee",
           variable = c("ir", "cot", "t.pr.ag", "t.br.ag"),
           sgn = "pos", agent = "etat")
po
```

```
# A tibble: 300 x 5
  an    cpte  variable agent valeur
* <chr> <chr> <chr>    <chr>  <dbl>
1 1949  expl  t.pr.ag  etat    1.60
2 1950  expl  t.pr.ag  etat    1.96
3 1951  expl  t.pr.ag  etat    2.54
# i 297 more rows
```

Ensuite on remplace les noms courts par des noms longs pour les variables en utilisant la fonction `add_label` :

```
po <- po |> add_label("variable")
```

Le graphique que l'on souhaite créer est un graphique à "surfaces cumulées". Il nécessite que les différents impôts soient mesurés en proportions :

```
po$part <- po$valeur /
    condsummary(po, "an", sum, dim = "table") * 100
```

On peut enfin réaliser la Figure 1 à l'aide de **tinyplot**.

```
tinyplot(part ~ an | variable, data = po,
         type = type_area(stack = TRUE),
         palette = "harmonic",
         xlab = "année",
         ylab = "Pourcentage",
         legend = list("bottom!", ncol = 1))
```

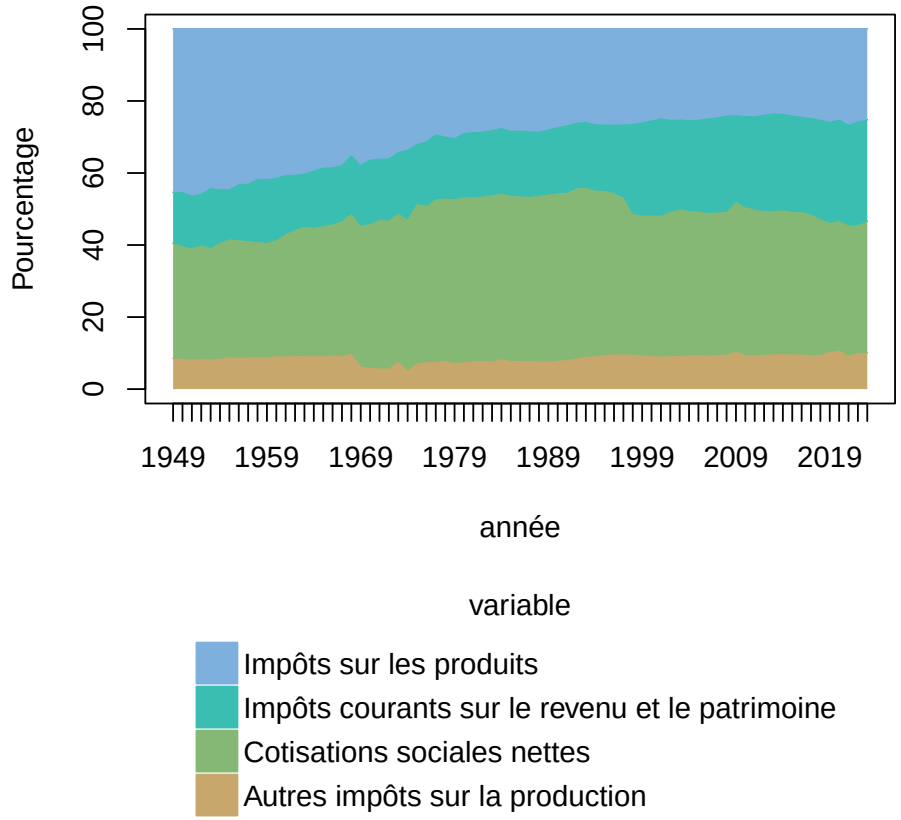


FIGURE 1 – Structure des prélèvements obligatoires en France

6.2 Parts des prélèvements obligatoire dans le PIB

Pour le montant des prélèvements obligatoires, il suffit de sommer les différents impôts précédemment extraits et de renommer la colonne :

```
po <- stbl(1949:2023, "tee",
           variable = c("ir", "cot", "t.pr.ag", "t.br.ag"),
           sgn = "pos", agent = "etat")
po_tot <- condsummary(po, "an", sum)
po_tot <- setaname(po_tot, "valeur", "impots")
```

Pour le PIB, on peut utiliser la définition basée sur la somme des demandes finales. On extrait les différentes composantes de la demande et on en exclut les consommations intermédiaires avant de les sommer pour chaque année et de renommer la colonne :

```
pib <- stbl(1949:2023, "produits", naf = "A1",
            variables = c("cf", "fbcf", "g", "x", "m", "cfisbl"))
pib$valeur[pib$variable == "m"] <- - pib$valeur[pib$variable == "m"]
pib <- condsummary(pib, "an", sum)
pib <- setaname(pib, "valeur", "pib")
```

On peut ensuite fusionner les deux tableaux et calculer la part des prélèvements obligatoires dans le PIB :

```
z <- merge(po_tot, pib)
z$part <- z$impots / z$pib * 100
tinyplot(part ~ an, z, type = "l", xlab = "année", ylab = "Pourcentage")
```

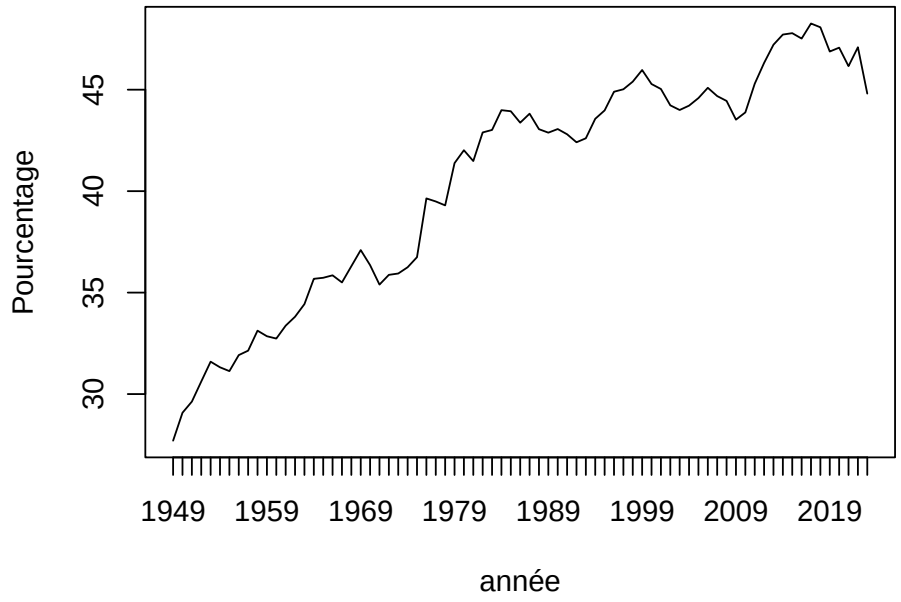


FIGURE 2 – Part des prélèvements obligatoires en France

6.3 Structure productive

On définit la structure par la part de valeur ajoutée produite par chaque secteur. On choisit une nomenclature simplifiée en 10 branches et on considère les années 1949 et 2023. La valeur ajoutée est la somme de la masse salariale et de l'excédent brut d'exploitation. On calcule ensuite les proportions sectorielles et on utilise un graphique en barres (Figure 3).

```
va <- stbl(c(1949, 2023), "branches", naf = "A10",
           variable = c("k", "l"))
va <- condsummary(va, c("an", "naf"), sum)
va$valeur = va$valeur / condsummary(va, "an", sum, dim = "table") * 100
tinyplot(valeur ~ naf, va, type = "barplot", facet = ~ an, flip = TRUE)
```

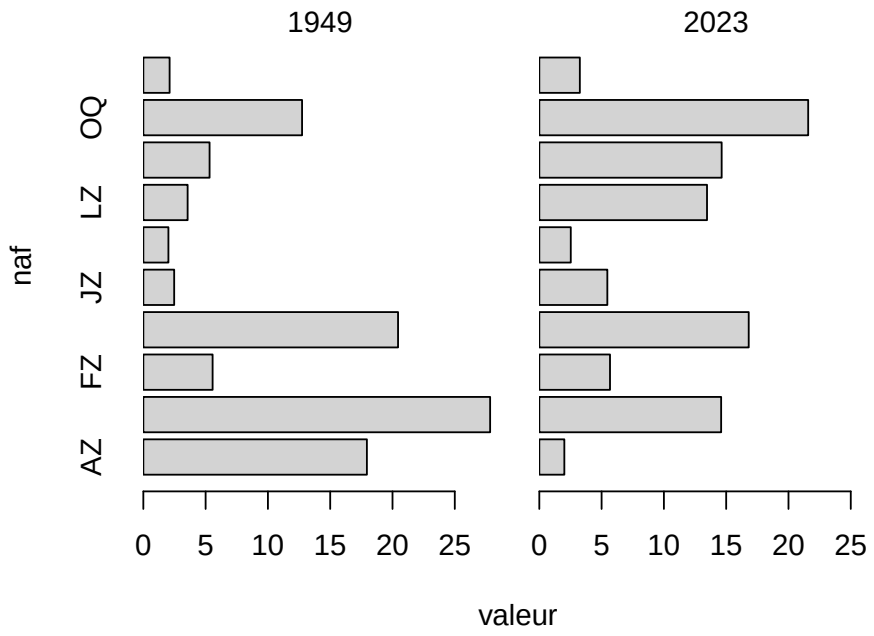


FIGURE 3 – Part de chaque secteur dans la valeur ajoutée

6.4 Revenus des facteurs par habitant en euros constants

On extrait de nouveaux les revenus des facteurs du compte des branches, on utilise pop et `ipc` pour déflater les valeurs et on remplace le nom court des facteurs par des noms long.

```
rev_facteurs <- stbl(1949:2023, "branches",
                     variables = c("l", "k"),
                     naf = "A1") |>
    pop() |>
    ipc(base = 2000) |>
    add_label("variable")
```

On utilise ensuite **tinyplot**, en indiquant dans la formule que l'on souhaite un type de ligne différent pour les deux facteurs :

```
tinyplot(valeur ~ an | variable, rev_facteurs,
         type = "l", lty = "by")
```

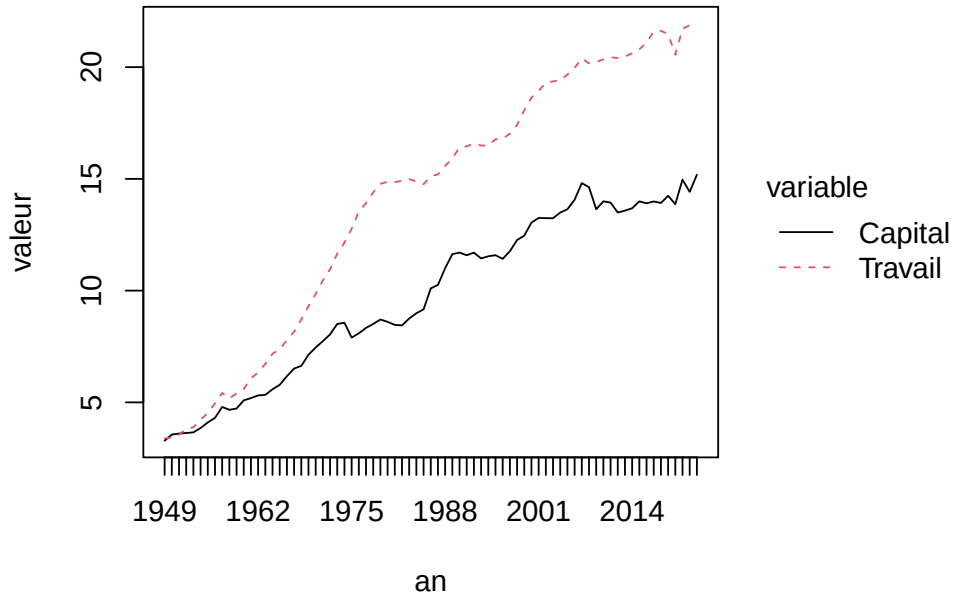


FIGURE 4 – Revenu du capital et du travail par habitant en euros de 2000